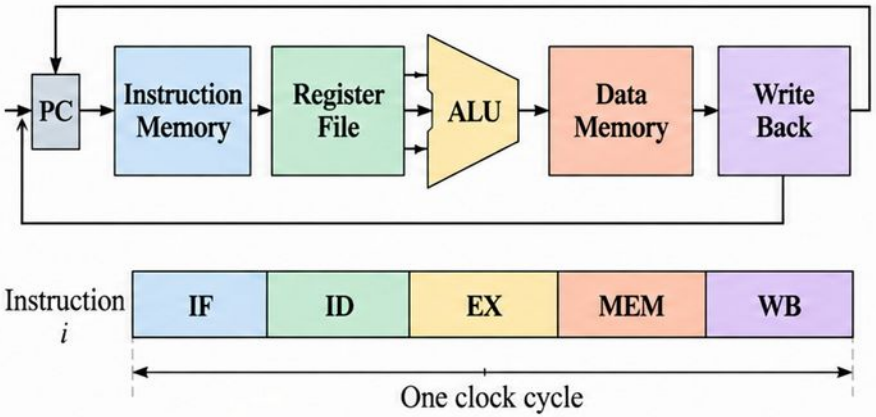
Chapter 4

Pipelining

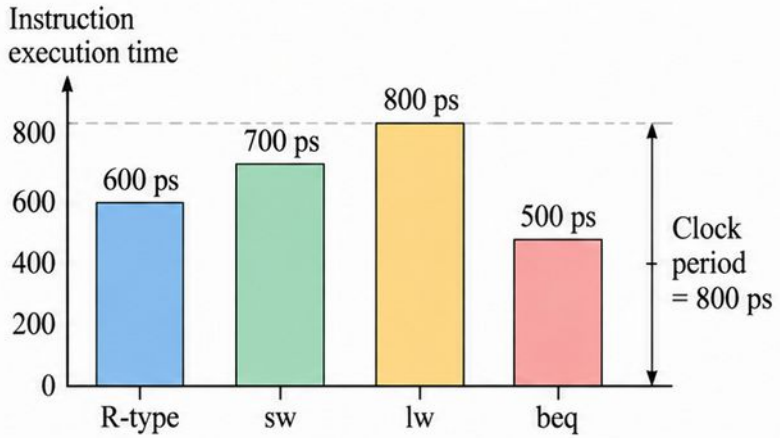
From Processor to Pipelining

Single-Cycle Processor



- One instruction completes all stages in one clock cycle.
- Clock period must be long enough to accommodate the slowest instruction (e.g., load).
- Only one instruction completes per cycle.

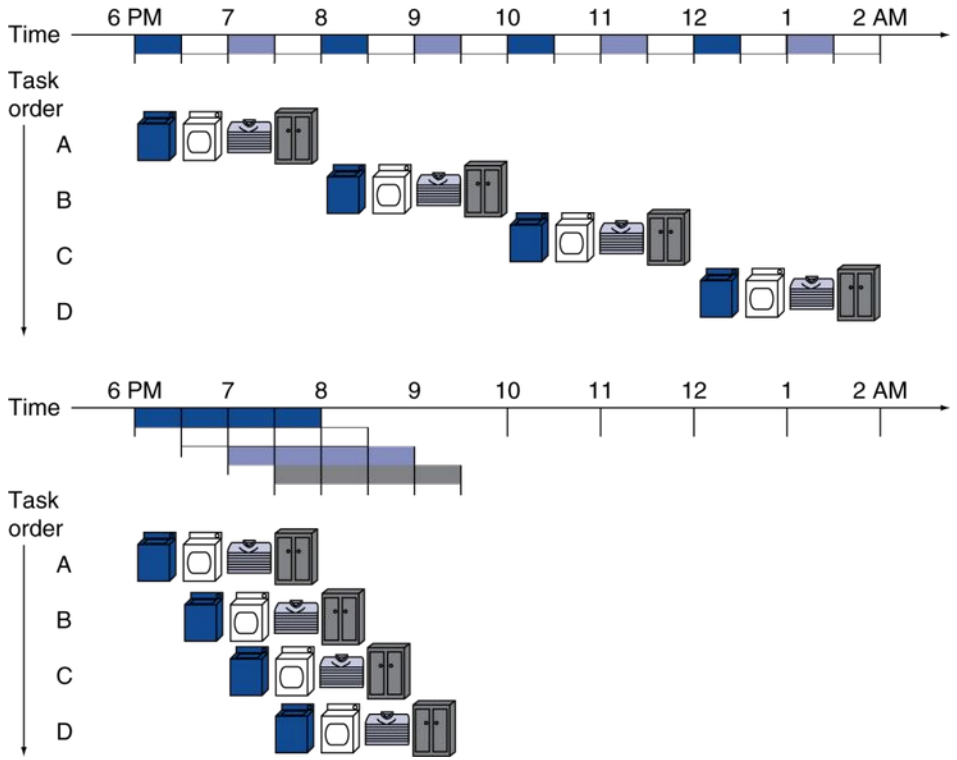
Performance Limitation



- Clock period is determined by the slowest instruction (e.g., load).
- All instructions, including shorter ones, must use the same long clock period (800 ps).
- This leads to inefficient use of hardware resources and limits performance.
- Only one instruction can complete per cycle.

Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance



- Four loads:
  - Speedup = 8/3.5 = 2.3
- Non-stop:
  - Speedup = 2n/0.5n + 1.5 ≈ 4 = number of stages

MIPS Pipeline

- Five stages, one step per stage
  1. IF: Instruction fetch from memory
  2. ID: Instruction decode & register read
  3. EX: Execute operation or calculate address
  4. MEM: Access memory operand
  5. WB: Write result back to register

What Happens in Each Stage?

Stage	Full Name	Main Operation	Example
IF	Instruction Fetch	Fetch instruction from instruction memory; update PC	PC → Instruction Memory
ID	Instruction Decode	Decode instruction and read source registers	Read $t1, $t2
EX	Execute	Perform ALU operation or calculate memory address	$t1 + 8
MEM	Memory Access	Read or write data memory when required	Memory[address]
WB	Write Back	Write result into destination register	$t0 ← result

Five Stages in Different Inst.

Stage	`add $t0,$t1,$t2`	`lw $t0,8($t1)`	`sw $t0,8($t1)`
IF	Fetch instruction	Fetch instruction	Fetch instruction
ID	Read **`$t1`**, **`$t2`**	Read **`$t1`**	Read **`$t1`**, **`$t0`**
EX	ALU addition	Calculate address	Calculate address
MEM	No data access	Read memory	Write memory
WB	Write **`$t0`**	Write **`$t0`**	No register write

Five Stages in Different Inst.

Q1. Consider the following MIPS instructions:

```
lw  $t0, 8($t1)
add $t2, $t0, $t3
sw  $t2, 12($t4)
```

For each instruction, identify the pipeline stage that performs the following operation:

(a) Calculates the effective memory address.

(b) Performs an arithmetic operation on register values.

(c) Writes a value to data memory.

A1.

(a) EX — for lw and sw.

(b) EX — for add.

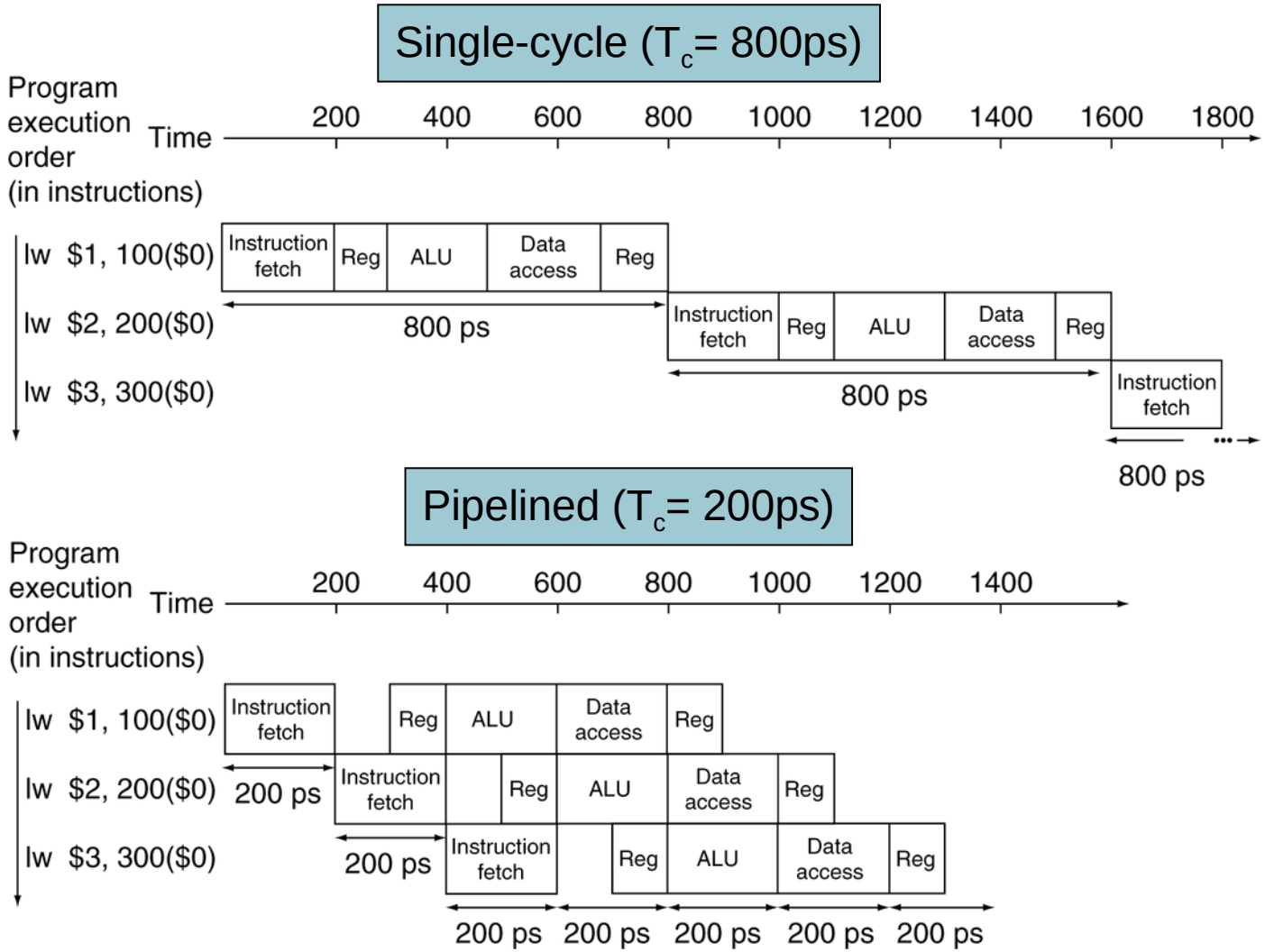
(c) MEM — for sw.

Pipeline Performance

- Assume time for stages is
  - 100ps for register read or write
  - 200ps for other stages
- Compare pipelined datapath with single-cycle datapath

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance



Pipeline Performance

Single-cycle (T_c= 800ps)

Q2. A five-stage MIPS pipeline has the following stage delays:

Stage	IF	ID	EX	MEM	WB
Delay (ps)	180	120	250	200	100

If the EX stage is optimized from 250 ps to 150 ps, what is the new pipeline clock cycle time, and how much does it improve?

A2.

Original clock cycle:

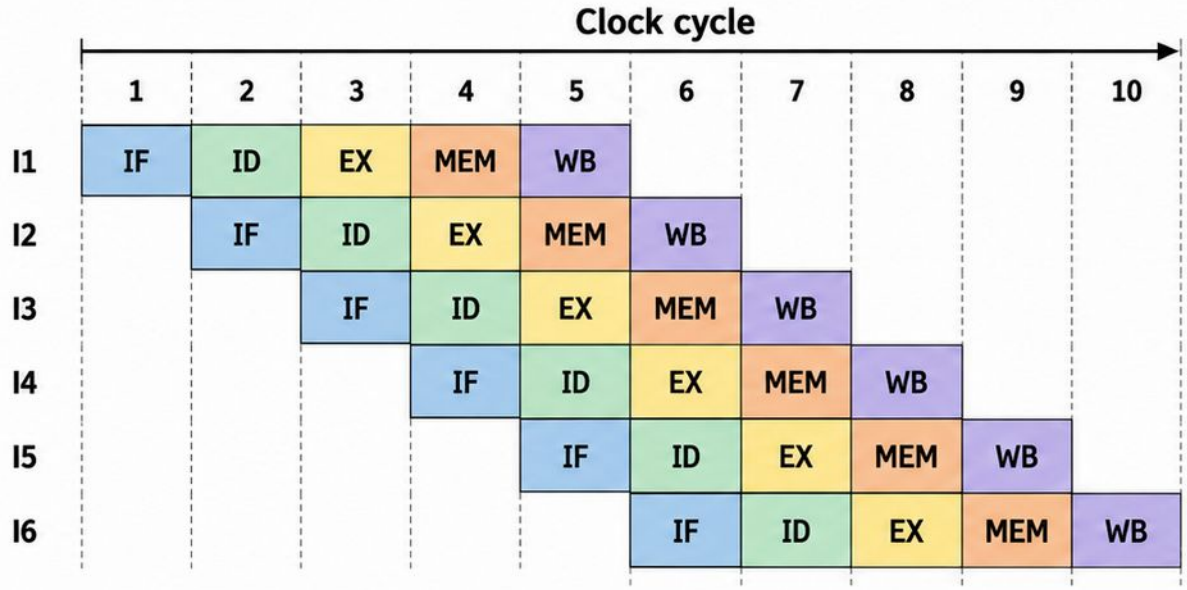
T_old = max(180,120,250,200,100) = 250 ps

New clock cycle: T_new=max(180,120,150,200,100)=200 ps

Clock period reduction:

(250-200)/250 = 20%

Pipeline Execution Time



Pipeline Execution Time

$$N_{\text{cycles}} = n + k - 1$$

$$T_{\text{execution}} = (n + k - 1)T_{\text{clock}}$$

- n : number of instructions
- k : number of pipeline stages
- T_{clock} : pipeline clock cycle time
- N_{cycles} : total number of clock cycles
- $T_{\text{execution}}$: total execution time

Example: 6 Instructions in a 5-Stage Pipeline

- Number of instructions: $n = 6$
- Number of stages: $k = 5$
- Clock cycle time: $T_{\text{clock}} = 200$ ps
- Assume no stalls

Total number of cycles:

$$N_{\text{cycles}} = 6 + 5 - 1 = \mathbf{10}$$

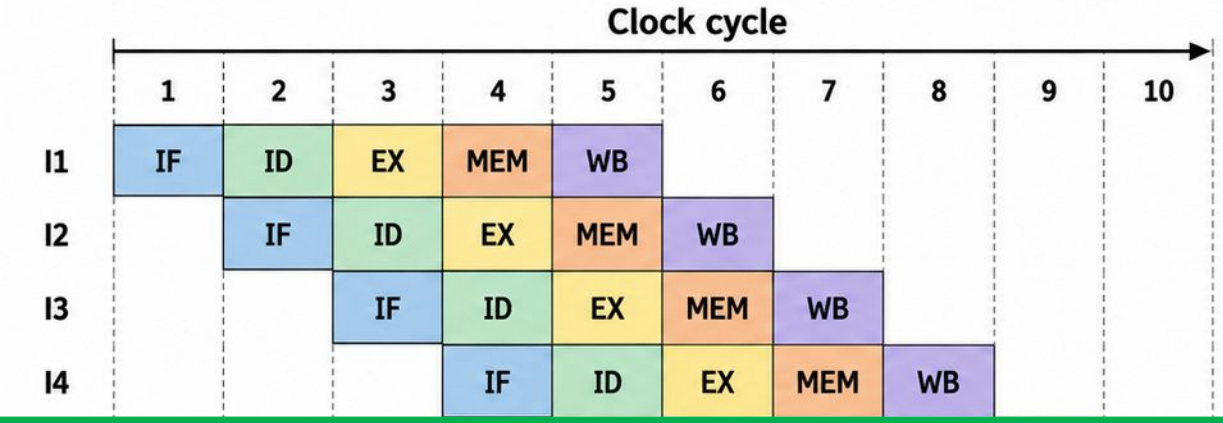
Total execution time:

$$T_{\text{execution}} = 10 \times 200 = \mathbf{2000\ ps}$$

Key Observations

- The first instruction completes after $k = 5$ cycles.
- After the pipeline is filled, one instruction completes per cycle.
- Executing n instructions requires $n + k - 1$ cycles (assuming no stalls).

Pipeline Execution Time



Pipeline Execution Time

$$N_{\text{cycles}} = n + k - 1$$

$$T_{\text{execution}} = (n + k - 1)T_{\text{clock}}$$

n : number of instructions

k : number of pipeline stages

Q3. A processor uses a 5-stage pipeline with a clock cycle time of 250 ps. A program completes in 4,500 ps without any stalls.

How many instructions were executed?

A3.

Total clock cycles:

N_cycles = 4500/250 = 18

Using the pipeline execution formula:

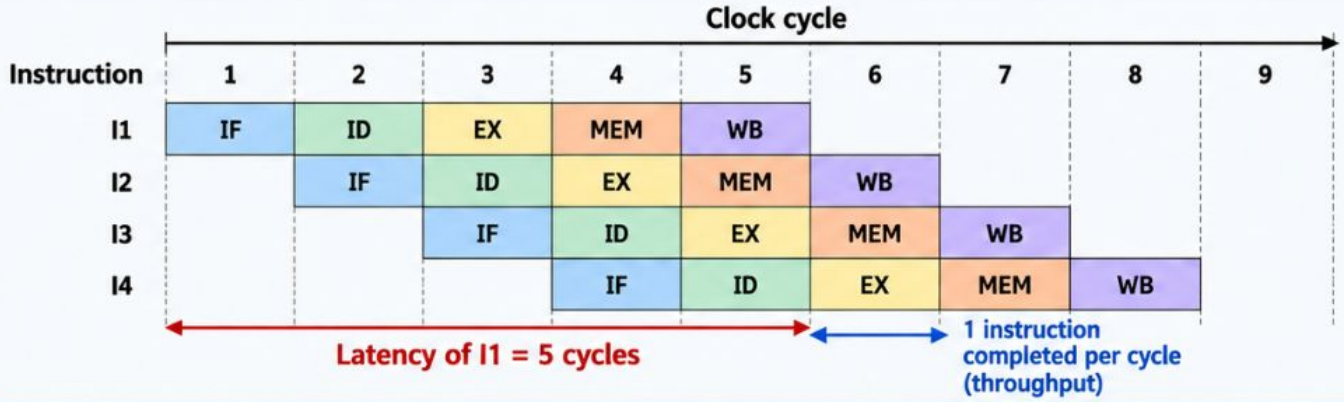
N_cycles = n + k – 1

18 = n + 5 – 1 => n = 14

Pipeline Speedup

- If all stages are balanced
  - i.e., all take the same time
  - Time between instructions$_{\text{pipelined}}$
    = $\dfrac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$
- If not balanced, speedup is less
- Speedup due to increased throughput
  - Latency (time for each instruction) does not decrease

Throughput vs. Latency



Pipeline stages (5-stage MIPS)

IF	Instruction Fetch
ID	Instruction Decode & Register Read
EX	Execute / ALU Operation
MEM	Data Memory Access
WB	Register Write

Throughput

- Number of instructions completed **per unit time**.
- In a pipeline, after it is filled, one instruction completes every clock cycle (assuming no stalls).
- Higher throughput improves overall performance.

$$\text{Throughput} = \frac{1}{T_{\text{clock}}} \quad \text{(instructions/second)}$$

Example ($T_{\text{clock}} = 200$ ps)

Throughput

$$= \frac{1}{200 \text{ ps}}$$

$$= 5 \times 10^9 \text{ instructions/s}$$

Latency

- Time required to complete one instruction (from start to finish).
- A k-stage pipeline requires k clock cycles.
- Pipelining **may increase instruction latency**, even though it improves throughput.

$$\text{Latency} = kT_{\text{clock}}$$

Example ($k = 5$, $T_{\text{clock}} = 200$ ps)

Latency

$$= 5 \times 200 \text{ ps}$$

$$= 1000 \text{ ps}$$

Single-cycle (for comparison)

Latency $= 800$ ps

Throughput vs. Latency

Q4. A processor is redesigned from a 5-stage pipeline with a 200 ps clock cycle to a 6-stage pipeline with a 150 ps clock cycle.Assuming no stalls, determine whether the instruction latency and ideal throughput improve.

A4.

Metric	5-Stage Pipeline	6-Stage Pipeline
Instruction Latency	5 × 200 = 1000 ps	6 × 150 = 900 ps
Ideal Throughput	5 × 10^9 instr/s	6.67 × 10^9 instr/s

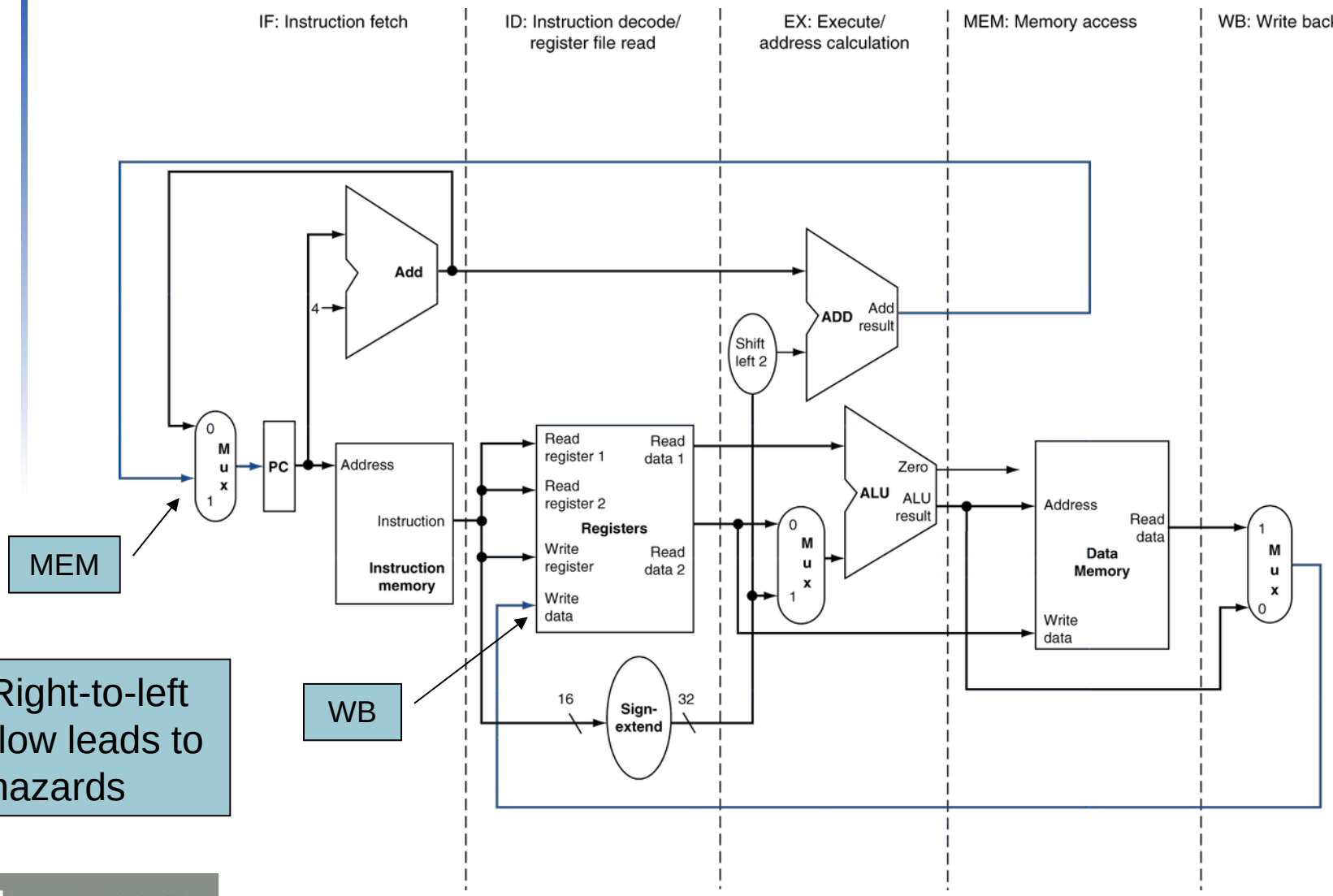
Latency: Decreases from 1000 ps to 900 ps (10% reduction).
Throughput: Increases from 5 to 6.67 billion instructions/s (approximately 33.3% increase).



Pipelining and ISA Design

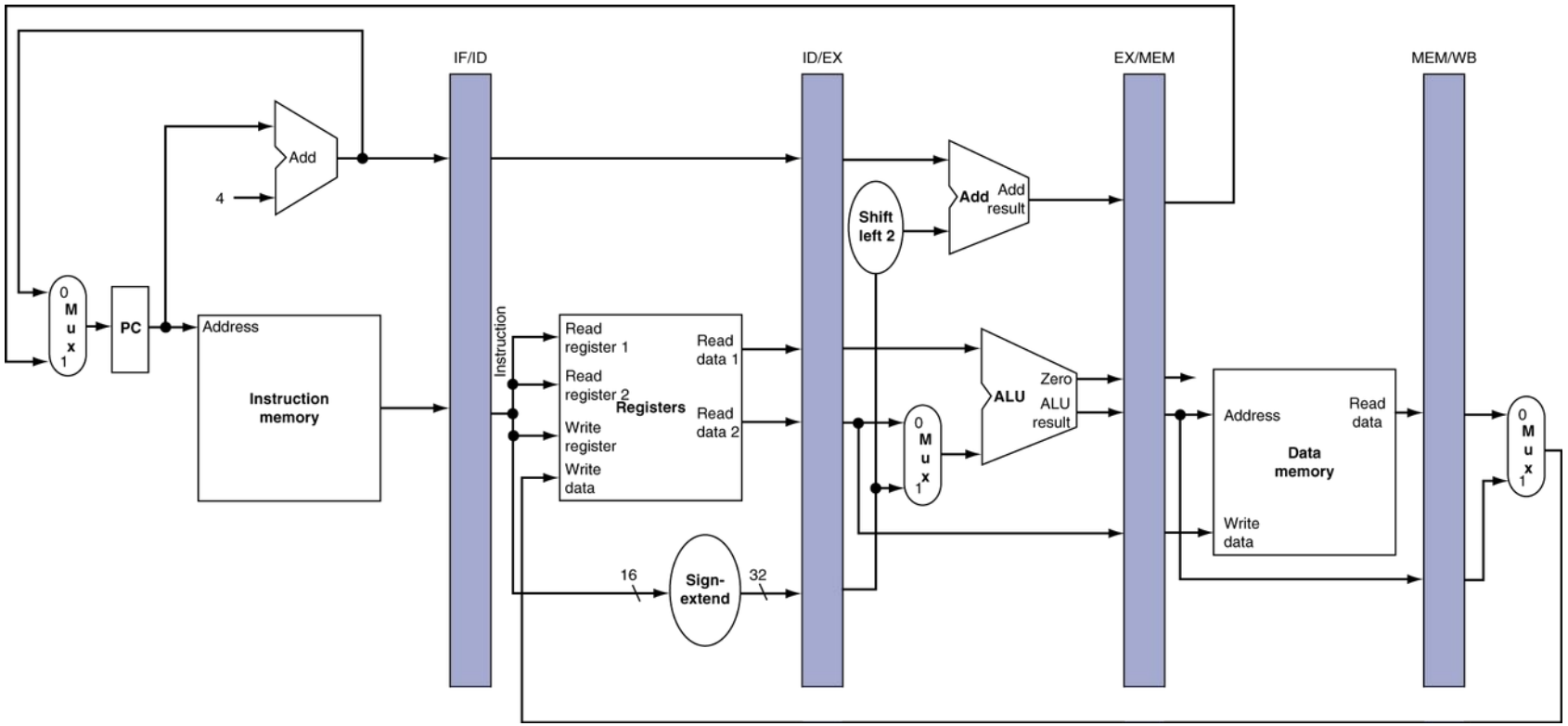
- MIPS ISA designed for pipelining
  - All instructions are 32-bits
    - Easier to fetch and decode in one cycle
    - c.f. x86: 1- to 17-byte instructions
  - Few and regular instruction formats
    - Can decode and read registers in one step
  - Load/store addressing
    - Can calculate address in 3rd stage, access memory in 4th stage
  - Alignment of memory operands
    - Memory access takes only one cycle

MIPS Pipelined Datapath



Pipeline registers

- Need registers between stages
  - To hold information produced in previous cycle



Pipeline registers

- Need registers between stages
  - To hold information produced in previous cycle

Q5. Consider the following MIPS instruction:
lw $t0, 12($t1)
Assume $t1 = 1000 and Memory[1012] = 75.When this instruction completes the MEM stage, what information must be preserved in the MEM/WB pipeline register for the WB stage?
A5.
Loaded data: 75
Destination register: $t0
Register write control: RegWrite = 1
The WB stage uses this information to write 75 into $t0.

Summary

- Five-Stage Pipeline: IF, ID, EX, MEM, WB.
- Pipeline Execution: Overlapping multiple instru ctions.
- Clock Cycle: Determined by the slowest stage.
- Performance: Execution time, speedup, and thr oughput.
- Pipeline Registers: Transfer data between stag es.

Section 4-1 Quick Questions

Q1. For lw, add, and sw, identify which stages perform address calculation, arithmetic operations, and memory writes.

Q2. If EX is optimized from 250 ps to 150 ps, what is the new clock cycle time and percentage reduction? (Stage delays: 180, 120, 250, 200, 100 ps)

Q3. A 5-stage pipeline with a 250 ps clock completes a program in 4500 ps. How many instructions were executed?

Q4. A processor changes from a 5-stage pipeline (200 ps) to a 6-stage pipeline (150 ps). How do latency and ideal throughput change?

Q5. For lw $t0, 12($t1), given $t1 = 1000 and Memory[1012] = 75, what information must the MEM/WB register preserve?

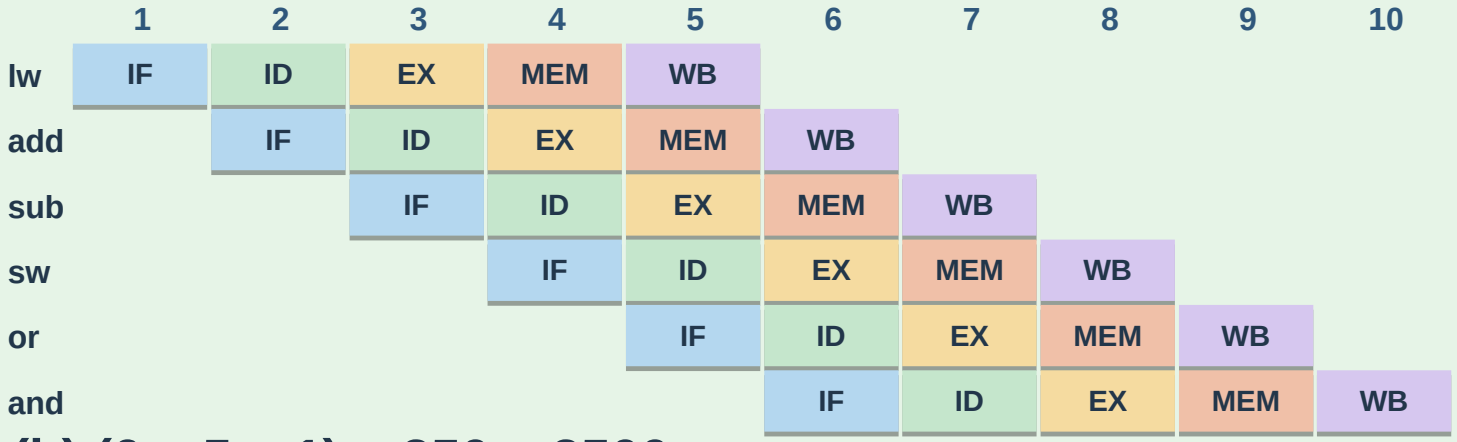
Exercise 1 — Pipeline Timing Analysis

A 5-stage pipeline executes six instructions (250 ps/cycle; no stalls):

```
lw  $t0, 0($t1)     add $t2, $t3, $t4     sub $t5, $t6, $t7
sw  $t2, 4($t1)     or  $t8, $t9, $s0     and $s1, $s2, $s3
```

(a) Draw timing diagram. (b) Find execution time.
(c) First full cycle: list all stages.

Answer



(b) (6 + 5 − 1) × 250 = 2500 ps

(c) Cycle 5: IF or, ID sw, EX sub, MEM add, WB lw

Key Point: The pipeline fills as instructions overlap across stages.

Exercise 2 — Pipeline Performance Comparison

Two processors execute the same program (no stalls):

Parameter	Processor A	Processor B
Pipeline stages	5	8
Clock period	220 ps	160 ps

(a) Time for 100 instructions?

(b) Speedup B/A? (c) Minimum n for B to be faster?

Answer

(a) Execution time for 100 instructions:

$T_A = (100 + 4) \times 220 = 22{,}880$ ps

$T_B = (100 + 7) \times 160 = 17{,}120$ ps

(b) Speedup = $22{,}880 / 17{,}120 \approx 1.34\times$

(c) $160(n + 7) < 220(n + 4) \Rightarrow 60n > 240$

$n > 4 \Rightarrow$ Minimum n = 5 instructions

Key Point: Shorter cycles can offset the fill/drain cost of deeper pipelines

Exercise 3 — Pipeline Register Error Analysis

Consider five instructions in a 5-stage pipeline (no stalls):

```
lw  $t2, 16($s1)
add $t3, $s2, $s3
sub $t4, $s4, $s5
or  $t5, $s6, $s7
and $t6, $s0, $s1
```

The destination register number of lw is omitted from EX/MEM.

(a) What goes wrong at WB? (b) Which registers must carry it?

(c) If WB wrongly uses the ID destination, which register is written?

Answer

(a) WB cannot reliably select $t2, even if the loaded data is correct.

(b) The destination number must pass through ID/EX → EX/MEM → MEM/WB.

(c) In cycle 5, lw is in WB and or is in ID; $t5 is wrongly selected.

Key Point: Pipeline registers keep each instruction's data and destination aligned.